

FOR XML PATH Consolidation

#sql-server #performance #set-based #refactoring

What the Pattern Looks Like

A query produces a result set in three parts via UNION (one branch per category), and each branch carries the same STUFF((... FOR XML PATH('')), 1, 1, '') correlated subquery to roll up child rows into a comma-separated list. The three FOR XML PATH expressions are textually identical except for which driver set they correlate to.

```
Select GRx.GroupType, GRx.GroupNum, ...,
       Stuff((Select N',' + 0.OrderId
              From OeOrder 0 ...
              Where 0.GroupNum = GRx.GroupNum
              And <large WHERE clause>
              Order By 0.OrderId
              For Xml Path(''), Type).value('.[1]', 'nVarChar(max)'), 1, 1, '') As Orders
From #BranchA GRx
Union
Select GRx.GroupType, ..., Stuff((Select ... For Xml Path ...) ...) From #BranchB GRx
Union
Select GRx.GroupType, ..., Stuff((Select ... For Xml Path ...) ...) From #BranchC GRx
```

If each branch carries 1,000 driver rows and the FOR XML PATH inner query touches 4 tables with combined row counts in the millions, the engine runs the same correlated lookup 3,000 times. The driver sets are different, but the inner work is the same.

Why It Hurts

Each FOR XML PATH expression is itself a correlated subquery. The engine evaluates it once per outer row. When the same expression appears in three UNION branches, you're running three separate sets of correlated lookups against the same underlying tables. The cost compounds across:

- Three execution-plan operators that each carry their own seek/scan cost.
- Three scans of the supporting indexes on OeOrder, OeOrderPoNumAssoc, OeOrderShipmentAssoc.
- Three independent evaluations of the WHERE clause logic, including any sub-expressions inside it.
- The UNION at the top forces an additional Sort + Stream Aggregate to dedup, even when the GroupType labels mean no row can ever appear in more than one branch.

In Query Store, this shows up as a query that touches the same tables many more times than its row count would suggest. STATISTICS IO will show reads on the inner tables that look proportional to (driver rows × branch count), not driver rows alone.

The Refactoring Steps

The fix has two halves: consolidate the driver sets first, then collapse the FOR XML PATH down to a single execution against the consolidated set.

1. Materialize all branches into one temp table via UNION ALL. Each branch already produces the same column shape (in our example, GroupType/GroupNum/PriInternal/DateFilled), so the union is straightforward. Use UNION ALL rather than UNION because the GroupType label guarantees the rows can't overlap.
2. Add an index to the consolidated temp table on the correlation column (the column the FOR XML PATH joins back on). A clustered index on the join key is usually sufficient.

3. Run the FOR XML PATH expression once against the consolidated temp table. The expression is structurally unchanged; only its outer driver source changes.
4. Drop the temp tables at the end.

```
-- Consolidated driver set (UNION ALL, distinct labels prevent overlap)
;With QualifyingGroups As
(
    Select 'Branch A' As GroupType, ... From ... Where ...
    Union All
    Select 'Branch B' As GroupType, ... From ... Where ...
    Union All
    Select 'Branch C' As GroupType, ... From ... Where ...
)
Select GroupType, GroupNum, PriInternal, DateFilled
Into #AllDrivers
From QualifyingGroups

-- One FOR XML PATH execution against the combined set
Select GRx.GroupType, GRx.GroupNum, GRx.PriInternal, GRx.DateFilled,
    Stuff((Select N',' + 0.OrderId
        From OeOrder 0 ...
        Where 0.GroupNum = GRx.GroupNum
        And <large WHERE clause>
        Order By 0.OrderId
        For Xml Path(''), Type).value('.[1]', 'nVarChar(max)'), 1, 1, '') As Orders
From #AllDrivers GRx
Order By GRx.DateFilled
```

Why This Works

You haven't changed the FOR XML PATH expression itself, and the per-row cost of executing it is unchanged. What you've changed is the multiplier in front of that cost. The engine now evaluates the inner correlated lookup once per total driver row across all branches, not once per driver row per branch.

If the three branches each had 1,000 rows and the FOR XML PATH inner cost was 100 logical reads per execution, the original was doing $3 \times 1,000 \times 100 = 300,000$ reads on the inner tables. The consolidated form does $3,000 \times 100 = 300,000$ reads, but only if the rows from all three branches survive into the final select. In the typical case, they all do, so the read count looks similar at the leaf level.

The wins are elsewhere:

- One execution plan operator instead of three, which means one set of cardinality estimates instead of three independent ones. Better estimates produce better join orders inside the FOR XML PATH expression.
- The UNION at the top is replaced by a UNION ALL inside the materialization, eliminating the dedup sort.
- The execution plan is shorter and easier to read, which matters for ongoing maintenance.
- The compiled plan reuses one set of statistics rather than three, reducing recompile risk on plan invalidation.

The biggest practical win usually comes from the cardinality estimate improvement on the consolidated form. The optimizer plans the inner lookup once, with the full cardinality of the consolidated driver set in view, and chooses the right join order for that cardinality. The branch-by-branch form forced three independent decisions on smaller driver sets, each of which could land on a different (and often worse) join order.

Watch Out for Query Hints on the Original Branches

Before you consolidate, scan each original SELECT for trailing `Option (...)` clauses. `Option (Maxdop 1)`, `Option (Recompile)`, `Option (Optimize For ...)`, and the rest of the `OPTION` family are statement-level hints. They can only sit at the end of a complete top-level statement. The moment you wrap a SELECT in a CTE branch, the hint has no legal place to live, and the consolidated query will fail to parse if you leave the hint inline.

You have three choices for translating each hint, ordered from closest-to-original to furthest:

1. Keep the hinted SELECT as its own standalone statement that creates the temp table, then `INSERT INTO ... SELECT ... UNION ALL ...` the other branches into it without hints. Preserves the original constraint precisely.
2. Move the hint onto the outer SELECT that materializes the CTE into the temp table. Broadens the scope of the hint to all branches. Acceptable when the additional branches are small and the broader hint scope does no harm.
3. Drop the hint. Only acceptable when you can articulate why the original reason for the hint no longer applies in the consolidated plan, and you can monitor for the absence after deployment.

The default move is option 1. Option 2 is a deliberate decision to expand scope, not a free simplification. Option 3 needs a written justification.

When NOT to Consolidate

- The branches produce different column shapes. If branch A returns 4 columns and branch B returns 6, you can't UNION ALL them without padding NULLs, and the FOR XML PATH expressions probably differ as well.
- The FOR XML PATH expressions are not actually identical. Read carefully, since sometimes the WHERE clause has a subtle difference per branch (a different filter on `OS.ShipmentId`, or a different status value). If they truly differ, consolidating loses the per-branch logic.
- The driver sets are tiny (under ~50 rows each). The consolidation overhead of an extra temp table outweighs the savings, and three small FOR XML PATH executions at 50 rows each are not worth optimizing.
- The branches have wildly different data distributions and the optimizer is choosing meaningfully different plans for each. In rare cases, three plans tuned independently can beat one plan tuned for the combined cardinality.

Recognizing the Pattern

Read the original SQL with this question in mind: "Is this FOR XML PATH expression textually identical across the UNION branches?" If yes, and the branches all produce the same column shape, this refactor applies.

In code review, the smell is repeated multi-line STUFF/FOR XML PATH blocks with only the source temp table or driver set changing between them. Anywhere you see three nearly-identical 12-line blocks, ask whether they should be one block running over a consolidated driver.

Related Concepts

- **Correlated Subqueries to CTEs**: the underlying principle. FOR XML PATH expressions are correlated subqueries that produce a string instead of a scalar.
- **UNION ALL Views**: the related principle for choosing UNION ALL over UNION when label columns prevent overlap.
- **Non-SARGable Predicates**: when the WHERE clause inside the FOR XML PATH has non-SARGable predicates, consolidating the driver set won't fix the per-row predicate cost.